🎤 Presentation Speech Script — English

Emotion-Aware Image Caption Generator

TeamML Newbies | ML Sessional Final Presentation

Target Duration: ~8 minutes

🟫 Slide 1 — Title (0:00–0:20)

> Good [morning/afternoon], everyone. I am [your name] from TeamML Newbies. Today, we are presenting our Machine Learning Sessional project — **Emotion-Aware Image Caption Generator**. Our team members are Anuron Maitro, Md. Asif Ali, and Himadri Gobinda Biswas. Let's get started.

🟫 Slide 2 — Introduction & Motivation (0:20–1:10)

> Image captioning is the task of automatically generating a text description for a given image. Standard captioning systems are already very powerful — they can tell us **what** is in an image: objects, people, actions.

> But they completely ignore one crucial dimension — **how the people in the image feel**.

> For example, given a photo of a person standing on a beach, a traditional model might generate: "*A person standing on a beach.*" That is factually correct, but it tells us nothing about whether this person is happy, sad, or worried.

> Our motivation comes from three areas. First, **accessibility** — visually impaired users benefit greatly from richer, more human-like descriptions. Second, **Human-AI interaction** — as AI becomes part of daily life, emotional awareness makes it more natural and empathetic. Third — fields like healthcare and social media where emotional context is critical.

> Our innovation is straightforward: we take a standard caption and enhance it. "*A person on a road*" becomes "*A joyful person on a road*" — by detecting the person's emotion and grammatically inserting it into the caption.

🟫 Slide 3 — Problem Statement & Approach (1:10–1:50)

> The core problem we address is: **How do we generate captions that describe both the visual content AND the emotional state of people?**

> This breaks down into three sub-challenges: generating an accurate base caption, detecting the facial emotion reliably, and merging emotion into the sentence naturally — without breaking grammar.

> Our solution uses three components working together: the **Vision Transformer + GPT-2** pair for caption generation, **DeepFace** for emotion recognition, and **NLTK's POS tagger** for grammatical emotion insertion. Let me now explain each component with the architecture diagrams.

▧ Slide 4 — Complete System Pipeline (1:50–2:30)

> This diagram shows our complete end-to-end pipeline. Let me walk you through it.

> When an image is uploaded, **two processes happen in parallel**. On the left branch, the image goes through the Vision Transformer encoder, which produces a 512-dimensional feature vector. That vector is passed to the GPT-2 decoder which generates the **base caption** — a plain text description.

> On the right branch, the **same image** is sent to DeepFace for emotion analysis. If a human face is detected, DeepFace returns the dominant emotion — like "happy" or "sad". If no face is found, the system defaults to "neutral".

> Both outputs — the base caption and the detected emotion — are then sent to Stage 3, the **NLTK Processor**, which grammatically inserts the emotion adjective before the first noun in the caption.

> The final output is the **Emotion-Aware Caption**.

▧ Slide 5 — ViT Encoder Architecture (2:30–3:20)

> Now let's look at the first major component — the Vision Transformer Encoder. This diagram shows exactly how image information flows through our encoder.

> The input image, which is 224×224 pixels with 3 colour channels, is first divided into **196 non-overlapping patches** of size 16×16. Each patch is projected to a **768-dimensional embedding vector** — and 768 comes directly from the formula: 16 squared times 3, which is 768.

> A special [CLS] token is prepended, and learned positional embeddings are added to each of the 197 tokens. This preserves spatial information since transformers have no built-in notion of position.

> These tokens then pass through **8 Encoder Blocks**. Each block has: Multi-Head Self-Attention with **4 heads**, a Feed-Forward MLP that expands to 3072 dimensions and contracts back to 768, followed by Layer Normalization and residual connections.

> Importantly, we use **vit_base_patch16_224**, a pre-trained ViT model from the `timm` library. The [CLS] token from the final block represents the entire image in 768 dimensions. This is then linearly projected down to **512 dimensions** to match the GPT-2 decoder's hidden size.

▧ Slide 6 — GPT-2 Decoder Architecture (3:20–4:10)

> The second major component is our custom GPT-2 style decoder. This diagram shows how caption tokens are generated.

> The decoder takes two inputs: the **image features from ViT** (512-dim), and the **previously generated tokens** starting from [BOS].

> Each of the **8 Decoder Blocks** contains three operations. First, **Causal Self-Attention** with 8 heads — this is masked so each token can only attend to previous tokens, maintaining the autoregressive

> property. Second, **Cross-Attention** — here each token queries the image features, so every word generated is grounded in what the image contains. Third, a Feed-Forward MLP.

> After all 8 blocks, a linear projection maps the 512-dim output to the full vocabulary of 50,260 tokens. The model picks the most likely next token — this is **greedy decoding** via argmax — and feeds it back as input. This continues until the [EOS] token is generated or the maximum length of 50 tokens is reached.

> The dashed red arrow in the diagram represents this **autoregressive loop**.

▧ Slide 7 — Emotion Detection (4:10–4:40)

> For emotion detection, we use **DeepFace** — a state-of-the-art pre-trained framework. We call it with `enforce_detection=False` so that it gracefully handles images where faces may be partially visible or at an angle.

> DeepFace classifies emotions into 7 categories: Happy, Sad, Angry, Surprise, Fear, Disgust, and Neutral. It returns a probability for each, and we extract the **dominant emotion**.

> If face detection completely fails, we default to "neutral" — so the pipeline always produces an output.

▧ Slide 8 — NLTK Insertion (4:40–5:10)

> The third component is our NLTK-based insertion algorithm. It is simple but effective.

> We tokenize the base caption and apply **Part-of-Speech tagging**. Then we find the first **noun** in the sentence — any token tagged NN, NNS, NNP, or NNPS — and insert the corresponding emotion adjective immediately before it.

> For example: "*a person standing on a road*". POS tagging identifies "person" as the first noun at position 1. Emotion is "happy" → adjective is "joyful". Result: "*a joyful person standing on a road.*"

> The emotion-to-adjective mapping is: happy→joyful, sad→melancholic, angry→angry, surprise→surprised, fear→fearful, disgust→disgusted, neutral→calm.

▧ Slide 9 — Dataset (5:10–5:30)

> We used the **Flickr30k dataset** — 8,091 images with 5 captions each. After deduplication (keeping one caption per image), we split the data 80/20 — approximately 6,473 images for training and 1,618 for testing. This split is done randomly at runtime. All images are resized to 224×224 and normalized with standard ImageNet mean and standard deviation values.

▧ Slide 10 — Training Configuration (5:30–6:00)

> Our training used 20 epochs total. In the first 2 epochs, the ViT encoder is **frozen** — only the GPT-2 decoder and the 768-to-512 projection layer are trained. This stabilizes early training.

> From epoch 3 to 20, we unfreeze all weights and train end-to-end. Learning rate is **2×10^{-4}**, batch size is **16**, and we use the **Adam optimizer** with weight decay of 10^{-6}. Dropout of 0.5 is applied to attention, MLP, and embedding layers throughout. We trained on a **Kaggle NVIDIA Tesla T4 GPU** for approximately 6 to 8 hours.

▧ Slide 11 — Results (6:00–6:40)

> Let's look at the quantitative results. On our test set of approximately 1,618 images, our model achieves: BLEU-1 of 33.04%, BLEU-2 of 16.23%, BLEU-3 of 9.13%, BLEU-4 of 5.28%, and a METEOR score of 22.59%.

> Compared to the reference paper — Sankhla et al., IEEE ACCAI 2024 — our BLEU scores are lower. However, this is expected because the reference paper uses a more fully fine-tuned pipeline with more training resources. Our METEOR score of 22.59% is within reasonable range of their 24.83%. Importantly, our qualitative results show that the emotion integration is meaningful and grammatically correct.

▧ Slides 12 & 13 — Qualitative Results (6:40–7:10)

> Here you can see sample outputs from our system. For a happy image, the base caption "*Two children playing in a park*" becomes "*Two joyful children playing in a park.*" For a sad image, "*A person sitting alone on a bench*" becomes "*A melancholic person sitting alone on a bench.*"

> For an angry image, we get an "angry person with crossed arms," and for a neutral image, a "calm businessman in an office." The emotion adjective is always inserted grammatically before the first noun.

▧ Slide 14 — Deployment (7:10–7:30)

> We deployed the system as a real web application. The backend is a FastAPI server on HuggingFace Spaces, running in a Docker container. The frontend is a React app hosted on Vercel. The model is stored separately in HuggingFace Models Hub because at 1.91 GB it exceeds HuggingFace's 1 GB storage limit for Spaces, and it is downloaded dynamically on startup. The entire deployment costs zero dollars per month.

▧ Slides 15 & 16 — Limitations, Conclusion (7:30–8:00)

> We acknowledge some limitations: the grammar can be slightly awkward with plural nouns, we capture only the dominant emotion, and the semantic fit of adjectives is not always perfect.

> In conclusion: we successfully built a complete 3-stage pipeline combining ViT, GPT-2, DeepFace, and NLTK. We achieved a METEOR score of 22.59% on Flickr30k and deployed a live, working web application. Our system demonstrates that emotionally-aware image descriptions are achievable with a combination of modern pre-trained models and lightweight rule-based NLP.

> Thank you very much. We are happy to take any questions.

[End of English Speech Script — Estimated: ~8 minutes]